

Roteirização de entregas urbanas por busca em grafo: largura, custo uniforme e A* com heurísticas admissível e superestimada

Miguel Da Silveira Palhares Leite, Pauline Fernandes Braz, Nathan David Oliveira da Rocha
Disciplina de Agentes e Busca (NI) — Professor: Aldo Henrique Mendes

Resumo. Modelamos a malha viária de uma cidade como um grafo dirigido, em que cruzamentos são estados, ruas são arcos com distância, ruas de mão única existem em um só sentido e ruas bloqueadas existem no mapa mas não podem ser usadas. Implementamos em Python, apenas com a biblioteca padrão, a busca em largura e uma busca melhor-primeiro genérica que produz custo uniforme, A* e gulosa. Comparamos as estratégias em quatro instâncias, uma delas sem solução, e estudamos o efeito de superestimar a heurística de linha reta com um fator k . Em instância pequena feita à mão, $k = 3$ levou a um caminho 60% mais caro; em malhas geradas com semente fixa, aumentar k reduziu os nós expandidos ao custo de uma perda pequena de qualidade.

Palavras-chave: busca heurística; algoritmo A*; admissibilidade; roteirização; espaço de estados.

1. Introdução

Na última milha de uma entrega, o veículo parte de um depósito e precisa chegar a um cruzamento de destino. Cada rua tem um comprimento e um sentido de circulação, e obras podem impedir o uso de uma delas. Escolher a rota de menor distância é um problema clássico de busca em espaço de estados: o mapa é conhecido, o efeito de seguir uma rua é certo, e o que falta é decidir a ordem em que os cruzamentos são explorados. Este trabalho implementa e compara estratégias de busca para esse problema e responde a uma pergunta específica do tema: *o que acontece com a otimalidade quando a heurística deixa de ser admissível?*

A Tabela 1 resume o agente pelo modelo PEAS. O ambiente é **totalmente observável** (o mapa e a lista de bloqueios são conhecidos antes do cálculo), de **agente único, determinístico** (seguir uma rua leva sempre ao mesmo cruzamento), **sequencial** (cada escolha afeta as seguintes), **estático** (o mapa não muda durante o cálculo), **discreto** (finitos cruzamentos e ruas) e **conhecido** (o efeito de cada ação está no mapa). Essas propriedades são uma premissa de modelagem: no trânsito real o ambiente seria dinâmico e parcialmente observável, e um planejador offline como o nosso precisaria replanejar.

Tabela 1. PEAS do agente roteirizador

Elemento	Descrição
Performance	menor distância total até o destino; nunca usar rua bloqueada nem andar contra o sentido; tempo de cálculo
Environment	malha viária: cruzamentos com coordenadas, ruas com distância, sentido e estado de bloqueio
Atuadores	escolher a próxima rua a partir do cruzamento atual; emitir a rota final
Sensores	mapa completo, cruzamento atual, coordenadas do destino, lista de bloqueios

A formulação tem cinco componentes: (1) **estados**, um cruzamento identificado por `str`; (2) **estado inicial**, a origem da instância; (3) **função sucessora**, que devolve os pares (vizinho, distância) das ruas que saem do cruzamento atual e não estão bloqueadas; (4) **teste de objetivo**, `estado == destino`; e (5) **custo de caminho**, a soma das distâncias das ruas percorridas, com custo de passo sempre positivo.

2. Fundamentação teórica

A **busca em largura** expande os estados em ordem crescente de profundidade, usando uma fila FIFO, e devolve o caminho com o menor número de passos, que só é o de menor custo quando todos os passos custam o mesmo (Russell e Norvig, 2022). A **busca de custo uniforme** expande sempre o estado de menor custo acumulado g e é equivalente ao algoritmo de Dijkstra (1959). Observamos que o enunciado do trabalho a trata como informada, enquanto Russell e Norvig (2022) a classificam como não informada, já que ela não usa conhecimento além da definição do problema; adotamos aqui a classificação do livro e a usamos como oráculo de otimalidade.

O A^* (Hart, Nilsson e Raphael, 1968) expande o estado de menor $f(n) = g(n) + h(n)$, em que h estima o custo restante. Se h é **admissível**, isto é, $h(n) \leq h^*(n)$ para todo n , com h^* o custo ótimo restante, o A^* devolve solução ótima. Se h é **consistente**, isto é, $h(n) \leq c(n, n') + h(n')$ para todo arco, o primeiro caminho encontrado até um estado já é o melhor, e nenhum estado precisa ser reaberto. Consistência implica admissibilidade, mas o contrário não vale. A busca **gulosa** usa apenas $f = h$ e não garante otimalidade. Por fim, multiplicar uma heurística admissível por um fator $k > 1$ equivale à busca ponderada estudada por Pohl (1970): o custo encontrado fica limitado a k vezes o ótimo, em troca de menos expansões.

3. Implementação

3.1 Representação do estado e do grafo

O estado é uma `str` com o identificador do cruzamento (por exemplo "A" ou "C_3_4"). Strings são imutáveis e *hashable*, então entram diretamente em `set` (descobertos) e como chaves de `dict` (`pais`, `melhor_g`). As coordenadas ficam fora do estado, em um `dict` à parte, porque não mudam durante a busca e só servem à heurística; põ-las dentro do estado o transformaria em tupla e encareceria o *hash* sem ganho algum. O grafo é uma lista de adjacência, `instancia["adjacencia"]`, com um dicionário de cruzamento para lista de pares (`vizinho`, `distância`). Para o cruzamento A da instância `il` ela vale `adjacencia["A"] = [("S", 1.5), ("B", 2.0), ("G", 3.2)]`. A escolha por lista em vez de matriz se justifica porque malhas viárias são esparsas, com 2 a 4 ruas por cruzamento, e uma matriz gastaria memória proporcional a $|V|^2$ quase toda vazia. Uma rua de mão dupla gera dois arcos e uma de mão única gera um; o bloqueio é guardado em `bloqueadas`, com os dois sentidos, e a rua bloqueada permanece na adjacência: quem a filtra é a função sucessora.

3.2 Função sucessora

```
1 def sucessores(instancia, estado):
2     # Função sucessora: ruas que saem de `estado`, exceto as bloqueadas.
3     bloqueadas = instancia["bloqueadas"]
4     return [(vizinho, d) for vizinho, d in instancia["adjacencia"][estado]
5             if (estado, vizinho) not in bloqueadas]
```

A primeira linha depois do comentário, `bloqueadas = instancia["bloqueadas"]`, pega o conjunto de arcos proibidos. A compreensão de lista percorre `instancia["adjacencia"][estado]`, que já contém somente arcos que *saem* de estado; assim o sentido de circulação está codificado na própria estrutura e ir contra a mão é impossível. A cláusula `if (estado, vizinho) not in bloqueadas` descarta as ruas em obras. Sem essa linha, o A–G da instância `il`, que mede 3,2, seria uma opção de rota, e o caminho devolvido poderia atravessar uma rua interdita.

3.3 Busca em largura

A fronteira é um `collections.deque`, e a retirada é `popleft()`, que custa $O(1)$; usar `lista.pop(0)` custaria $O(n)$ e transformaria a busca em quadrática. O teste de objetivo é feito na geração, o que é correto na largura porque o primeiro caminho que chega ao destino já tem o menor número de passos.

```
BUSCA-EM-LARGURA(problema):
    fronteira ← deque([inicial]); descobertos ← {inicial}; pais ← {inicial: nulo}
    enquanto fronteira não vazia:
        estado ← fronteira.popleft()                                # FIFO
        para cada (vizinho, d) em SUCESSORES(estado):
            se vizinho ∉ descobertos:
                descobertos ← descobertos ∪ {vizinho}; pais[vizinho] ← estado
                fronteira.append(vizinho)
            se vizinho = destino: retorne RECONSTRUIR(pais, vizinho)  # teste na geração
    retorne FALHA
```

```

1 while fronteira:
2     estado = fronteira.popleft()
3     if not metricas.contar_expandido(estado, fronteira, limite_expansoes):
4         return _encerrar("limite_atingido", metricas)
5     for vizinho, _ in sucessores(instancia, estado):
6         if vizinho not in descobertos:
7             descobertos.add(vizinho)
8             pais[vizinho] = estado
9             fronteira.append(vizinho)
10            metricas.contar_gerado(fronteira, vizinho)
11            if vizinho == destino:
12                return _sucesso(instancia, pais, vizinho, metricas)

```

Na linha 1, o laço roda enquanto houver estados na fronteira. A linha 2 retira o estado mais antigo (FIFO). A linha 3 chama `metricas.contar_expandido`, que conta a expansão e devolve `False` se o limite de expansões foi atingido; nesse caso a linha 4 devolve o status "limite_atingido". A linha 5 percorre os pares devolvidos por `sucessores`. A linha 6 é o controle de descobertos: só entra na fronteira o vizinho ainda não visto, e as linhas 7 e 8 registram-no em descobertos e gravam em `pais` de onde ele veio. A linha 9 o coloca no fim da fila, a linha 10 conta a geração e atualiza o tamanho máximo da fronteira, e a linha 11 devolve o resultado assim que o destino é gerado.

O controle de descobertos é essencial neste domínio. Toda rua de mão dupla é um ciclo de dois arcos ($S \rightarrow A \rightarrow S \rightarrow A \dots$); sem descobertos, a fronteira acumularia cópias dos mesmos cruzamentos e a busca cresceria exponencialmente, e na instância sem solução ela jamais terminaria, pois o destino nunca seria alcançado e os ciclos alimentariam a fila indefinidamente.

3.4 Busca melhor-primeiro

Uma única função, `busca_melhor_primeiro(instancia, prioridade)`, implementa custo uniforme ($\text{prioridade} = g$), A^* ($g + h$) e gulosa (h) por meio de três invólucros de uma linha. A decisão deixa dois laços principais para explicar em vez de quatro e faz da estratégia uma única expressão. A fronteira é uma lista mantida como heap por `heapq`, com entradas (`f`, contador, `g`, estado). O contador, obtido de `itertools.count`, é o desempate: sem ele, dois estados com o mesmo f levariam o Python a comparar os estados entre si, o que é arbitrário. O `heapq` não tem *decrease-key*; usamos remoção preguiçosa: quando um caminho melhor é achado, a entrada nova é inserida e a antiga é descartada ao ser retirada, comparando seu g com `melhor_g[estado]`.

```

BUSCA-MELHOR-PRIMEIRO(problema, prioridade):
    fronteira ← heap[(prioridade(inicial, 0), 0, 0, inicial)]
    melhor_g ← {inicial: 0}; pais ← {inicial: nulo}
    enquanto fronteira não vazia:
        (_, _, g, estado) ← heappop(fronteira)
        se g > melhor_g[estado]: continue # entrada obsoleta
        se estado = destino: retorne RECONSTRUIR(pais, estado) # teste na retirada
        para cada (vizinho, d) em SUCESSORES(estado):
            g_novo ← melhor_g[estado] + d
            se vizinho ∉ melhor_g ou g_novo < melhor_g[vizinho]:
                melhor_g[vizinho] ← g_novo; pais[vizinho] ← estado
                heappush(fronteira, (prioridade(vizinho, g_novo), contador++, g_novo, vizinho))
    retorne FALHA

```

```

1 while fronteira:
2     _, _, g, estado = heapq.heappop(fronteira)
3     if g > melhor_g[estado]:
4         continue
5     if estado == destino:
6         return _sucesso(instancia, pais, estado, metricas)
7     if not metricas.contar_expandido(estado, fronteira, limite_expansoes):
8         return _encerrar("limite_atingido", metricas)
9     for vizinho, d in sucessores(instancia, estado):
10        g_novo = melhor_g[estado] + d
11        if vizinho not in melhor_g or g_novo < melhor_g[vizinho]:
12            melhor_g[vizinho], pais[vizinho] = g_novo, estado
13            entrada = (prioridade(vizinho, g_novo), next(contador), g_novo, vizinho)
14            heapq.heappush(fronteira, entrada)
15            metricas.contar_gerado(fronteira, entrada)

```

A linha 2 retira a entrada de menor prioridade e a desempacota em g e estado. As linhas 3 e 4 descartam a entrada obsoleta: se o g guardado é maior que `melhor_g[estado]`, um caminho melhor já foi encontrado depois que aquela entrada foi inserida. A linha 5 faz o teste de objetivo **na retirada**, e não na

geração; testar na geração quebraria a otimalidade, pois o primeiro caminho gerado até o destino pode ser mais caro que outro ainda na fronteira. Só depois disso, na linha 7, a expansão é contada: entradas obsoletas e o objetivo retirado não são expansões. A linha 10 calcula o custo do vizinho como $\text{melhor_g}[\text{estado}] + d$, dentro do laço de vizinhos, porque d depende da rua específica. Se o vizinho é novo ou o caminho é melhor (linha 11), a linha 12 atualiza melhor_g e pais , a linha 13 monta a tupla com a prioridade e o contador, e as linhas 14 e 15 inserem-na e contam a geração. Aqui melhor_g faz o papel de descobertos: sem ele o mesmo problema dos ciclos de mão dupla reapareceria.

3.5 Reconstrução do caminho e métricas

```
1 def reconstruir_caminho(pais, estado_final):
2     # Segue `pais` do objetivo até a origem e inverte, devolvendo a rota em ordem.
3     caminho = []
4     while estado_final is not None:
5         caminho.append(estado_final)
6         estado_final = pais[estado_final]
7     return caminho[::-1]
```

A linha 3 cria a lista; o laço da linha 4 anda de estado_final até a origem, cujo pai é None , acrescentando cada estado (linha 5) e subindo pelo dicionário pais (linha 6); a linha 7 inverte a lista para a ordem origem→destino. A alternativa, guardar o caminho inteiro em cada nó da fronteira, custaria $O(\text{profundidade})$ de memória por nó; o dicionário pais custa uma entrada por estado alcançado, $O(|V|)$ no total.

As métricas ficam em um objeto *Metricas* com dois métodos que cabem em uma linha cada no laço, o que mantém os trechos dentro do limite de 15 linhas. Todas as buscas devolvem um *ResultadoBusca*(caminho , custo , status , metricas) com status em {sucesso, sem_solucao, limite_attingido}.

3.6 Heurísticas

h_linha_reta devolve a distância euclidiana do cruzamento ao destino. **Prova de admissibilidade:** qualquer caminho de n ao destino é uma sequência de ruas; cada rua mede ao menos a linha reta entre suas pontas, o que validar_instancia garante; pela desigualdade triangular, a soma dessas linhas retas é ao menos a linha reta de n ao destino; logo $h(n) \leq h^*(n)$. A mesma condição dá a consistência, pois $h(n) \leq c(n, n') + h(n')$ vale para cada rua. A prova depende da validação: uma rua cadastrada com distância menor que a linha reta é recusada com *ValueError*, e o gerador arredonda as distâncias *para cima* por essa razão. A função $\text{fabricar_h_superestimada}(k)$ devolve $h(n) = k \cdot \text{h_linha_reta}(n)$. Para $k = 3$ ela não é admissível: na instância $i1$, $h^*(A) = 2,0 + 1,5 = 3,5$, mas $3 \cdot 3,16 = 9,49 > 3,5$.

3.7 Decisões que poderiam ter sido outras

- **Lista × matriz de adjacência:** lista, pela esparsidade da malha.
- **dict pais × caminho no nó:** pais, por memória $O(|V|)$ contra $O(\text{profundidade})$ por nó.
- **Remoção preguiçosa × decrease-key:** preguiçosa, porque *heapq* não oferece a segunda; o preço são entradas obsoletas no heap.
- **Função genérica × uma por estratégia:** genérica; as estratégias diferem em uma expressão.
- **Teste na geração × na retirada:** geração na largura (correto e mais cedo); retirada em custo uniforme, A^* e gulosa (necessário à otimalidade).
- **Distância × tempo como custo:** distância; com tempo, a heurística admissível passaria a ser a linha reta dividida pela velocidade máxima.

4. Metodologia

Usamos quatro instâncias (Tabela 2). A $i1$ tem 7 cruzamentos e foi desenhada à mão para servir de oráculo: seus caminhos e custos foram calculados manualmente e viram os *asserts* de *testes.py*; ela contém uma avenida sinuosa de mão única ($M \rightarrow G$, 6,0, contra 2,0 em linha reta), uma rua bloqueada ($A \rightarrow G$) e uma rua de mão única que sai do destino ($G \rightarrow D$). As demais são grades geradas por *gerador.py* com *random.Random*(semente), espaçamento de 100 m com deslocamento aleatório de ± 10 m, distância igual à linha reta vezes um fator uniforme em $[1,0; 1,3]$ arredondada para cima, 30% de ruas de mão única e 10% de bloqueios. A semente 42 da $i2$ não produziu caminho entre origem e destino (verificado pela busca em largura), então usamos a semente 43. A $i4$ foi construída para ter solução impossível por razão de domínio:

o cruzamento de destino só tem ruas de mão única que *saem* dele. As instâncias são salvas em JSON e o experimento lê os arquivos.

Tabela 2. Instâncias usadas

Instância	Cruzamentos	Ruas	Mão única	Bloqueadas	Semente
i1_pequena	7	9	2	1	—
i2_media	36	60	21	7	43
i3_grande	400	760	251	58	42
i4_sem_solucão	100	180	53	15	42

Comparamos largura, custo uniforme, A* com h (linha reta), A* com $h \times 2$ e $h \times 3$, e gulosa. As métricas seguem definições fechadas: **gerados** são as inserções na fronteira, incluindo o estado inicial (reinsserir um estado com g menor conta de novo, e na largura o objetivo entra na fronteira antes do retorno); **expandidos** são os estados retirados da fronteira cujos sucessores foram gerados (entradas obsoletas e o objetivo retirado não contam); **custo** é a soma das distâncias do caminho devolvido; **passos** é o número de ruas; **tempo** é a mediana de 5 execuções medidas com `time.perf_counter`; e **máx. fronteira** é o maior tamanho da fronteira. A coluna “ótimo?” compara o custo com o do custo uniforme, usado como oráculo. Para a exigência extra, variamos k em $\{1,0; 1,5; 2,0; 3,0; 5,0\}$ nas instâncias com solução. Ambiente: Python 3.13.14 | Windows-11-10.0.26300-SP0 | processador: AMD64 Family 25 Model 33 Stepping 2, AuthenticAMD | repetições por medida: 5. Todos os números foram gerados por uma única execução de `experimentos.py`.

5. Resultados

A Tabela 3 traz as métricas medidas. Na i1 os resultados coincidem com o cálculo manual: a largura devolve $S \rightarrow M \rightarrow G$ (2 passos, custo 8,0); o custo uniforme e o A* devolvem $S \rightarrow A \rightarrow B \rightarrow G$ (3 passos, custo 5,0); o A* com $h \times 3$ devolve $S \rightarrow M \rightarrow G$, com custo 8,0. A Figura 1 mostra o grafo da i1 com os dois caminhos.

Tabela 3. Métricas por instância e estratégia (tempo: mediana de 5 execuções)

Instância	Estratégia	Custo	Passos	Gerados	Expand.	Máx. front.	Tempo (ms)	Ótimo?
i1_pequena	Largura	8,0	2	5	2	3	0,005	não
i1_pequena	Custo uniforme	5,0	3	8	6	3	0,009	sim
i1_pequena	A* (h)	5,0	3	8	5	3	0,011	sim
i1_pequena	A* ($h \times 2$)	5,0	3	7	4	3	0,010	sim
i1_pequena	A* ($h \times 3$)	8,0	2	5	2	3	0,007	não
i1_pequena	Gulosa (h)	8,0	2	5	2	3	0,006	não
i2_media	Largura	1111,7	10	36	34	8	0,028	sim
i2_media	Custo uniforme	1111,7	10	40	35	8	0,041	sim
i2_media	A* (h)	1111,7	10	35	28	11	0,042	sim
i2_media	A* ($h \times 2$)	1121,4	10	20	11	9	0,023	não
i2_media	A* ($h \times 3$)	1121,4	10	20	11	9	0,023	não
i2_media	Gulosa (h)	1121,4	10	20	10	10	0,021	não
i3_grande	Largura	4329,2	38	398	397	24	0,313	não
i3_grande	Custo uniforme	4093,7	38	434	397	28	0,501	sim
i3_grande	A* (h)	4093,7	38	374	304	37	0,486	sim
i3_grande	A* ($h \times 2$)	4228,3	38	84	40	44	0,090	não
i3_grande	A* ($h \times 3$)	4259,3	38	85	39	46	0,089	não
i3_grande	Gulosa (h)	4409,5	38	79	38	41	0,078	não
i4_sem_solucão	Largura	—	—	95	95	11	0,068	—
i4_sem_solucão	Custo uniforme	—	—	105	95	12	0,107	—
i4_sem_solucão	A* (h)	—	—	114	95	20	0,136	—
i4_sem_solucão	A* ($h \times 2$)	—	—	237	195	38	0,311	—
i4_sem_solucão	A* ($h \times 3$)	—	—	371	322	39	0,495	—
i4_sem_solucão	Gulosa (h)	—	—	542	502	26	0,704	—

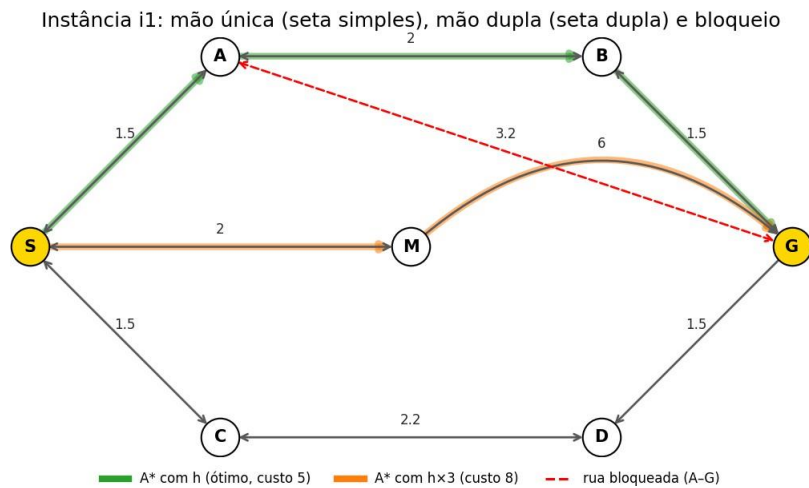


Figura 1. Instância i1: caminho ótimo do A* (verde) e caminho do A* com $h \times 3$ (laranja); a rua A–G, tracejada, está bloqueada

A Figura 2 mostra a varredura de k . Na i1, o custo é ótimo para $k \leq 2$ e passa a 8,0 para $k \geq 3$ (razão 1,60 do ótimo), com os expandidos caindo de 5 para 2. Na i2, o custo já sobe com $k = 1,5$ (razão 1,0087) e os expandidos caem de 28 para 10 em $k = 5$. Na i3, o A* com $k = 1$ expande 304 estados; com $k = 3$ expande 39, com custo 1,040 vezes o ótimo.

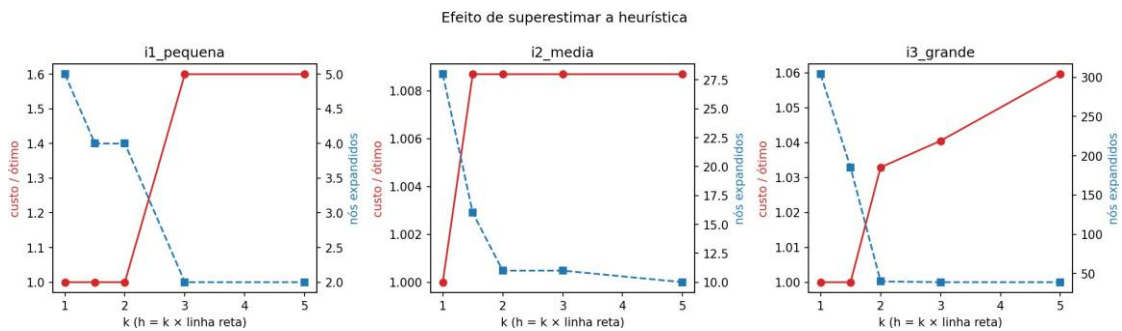


Figura 2. Varredura de k : custo/ótimo (vermelho, eixo esquerdo) e nós expandidos (azul, eixo direito)

6. Discussão

Largura: menos passos, não menor distância. Na i1 a largura acha o caminho de 2 passos e custo 8,0, contra 3 passos e custo 5,0: ela otimiza profundidade, e a avenida $M \rightarrow G$, longa, custa o mesmo “passo” que uma rua curta. Na i3 o número de passos é o mesmo (38), mas o custo da largura é 4329,2, 5,8% acima do ótimo de 4093,7.

A* expande menos que o custo uniforme com o mesmo custo. Na i3, o A* com h expande 304 estados contra 397 do custo uniforme (23% menos) e devolve o mesmo custo, pois a linha reta é admissível e consistente e orienta a busca ao destino. O ganho em tempo foi menor que o ganho em expansões (0,486 ms contra 0,501 ms), porque cada expansão do A* também calcula uma distância euclidiana; medidas abaixo de 1 ms devem ser lidas com cautela, pois estão perto da resolução do relógio e sujeitas a ruído.

Superestimar tira a garantia, não necessariamente o acerto. Na i1, $h \times 2$ ainda devolve o ótimo, e $h \times 3$ não. O limiar pode ser calculado: o A* só retira G pelo caminho ruim (custo 8) antes de olhar A se $f(A) = 1,5 + k \cdot 3,16 > 8$, isto é, $k > 2,06$; o teste automatizado confirma que $k = 2,05$ acerta e $k = 2,1$ erra. Dizer que “superestimar sempre quebra” seria falso; o correto é que a otimalidade deixa de ser garantida. O limite de Pohl (1970), custo $\leq k \cdot$ ótimo, é frouxo na prática: com $k = 5$ a i3 ficou em 1,060 vezes o ótimo, e a pior razão observada foi 1,60, na i1. O compromisso entre tempo e qualidade é real: 87% menos expansões na i3 por 4,0% de custo a mais.

Sem solução, a heurística inconsistente sai cara. Na i4, a largura, o custo uniforme e o A* com h expandem 95 estados, todos os alcançáveis, e concluem sem_solucao. Já o A* com $h \times 3$ expande 322: mais

do que o número de estados alcançáveis, o que só é possível porque estados são *reabertos* quando um caminho melhor aparece, exatamente o que a consistência evita.

Memória. Em malhas urbanas reais, com milhões de cruzamentos, o limite prático é a memória da fronteira. Em nossas instâncias pequenas, porém, o maior tamanho da fronteira foi *maior* no A* (37 na i3) que na largura (24), em parte por causa das entradas obsoletas do heap; portanto nossos dados não sustentam a afirmação de que a largura esgota a memória primeiro, que depende do crescimento assintótico da fronteira. Como trabalho futuro, o A* bidirecional e hierarquias de contração são as técnicas usuais para escalar esse problema. Uma limitação adicional é o ambiente estático assumido: bloqueios que aparecem durante a viagem exigiriam replanejamento.

7. Conclusão

Modelamos a roteirização urbana como busca em grafo dirigido e implementamos, sem bibliotecas de busca, a largura e uma busca melhor-primeiro genérica que produz custo uniforme, A* e gulosa. A heurística de linha reta é admissível e consistente, e o A* com ela igualou o custo do oráculo em todas as instâncias com solução, expandindo menos estados. Superestimar a heurística reduz as expansões mas perde a garantia de otimalidade, com um limiar exato ($k \approx 2,06$) na instância feita à mão e perdas pequenas nas malhas maiores. Como limitações, o ambiente é estático, as instâncias são sintéticas e as medidas de tempo são de sub-milissegundo; como trabalho futuro, propomos dados de mapas reais, custo em tempo de viagem e busca bidirecional.

Referências

- DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, p. 269–271, 1959.
- HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, v. 4, n. 2, p. 100–107, 1968.
- POHL, I. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, v. 1, n. 3–4, p. 193–204, 1970.
- RUSSELL, S.; NORVIG, P. *Inteligência Artificial: uma abordagem moderna*. 4. ed. Rio de Janeiro: LTC, 2022.